```
3. R = RealField(200)
4. print(R(1) / R(3))
5. print(R(0.1) + R(0.2))
6. print(R("0.1") + R("0.2"))
7. print(R.pi( ))
8. R = RealField(100, rnd='RNDN')
9. print(R(1)/3)
10. R = RealField(100, rnd='RNDD')
11. print(R(1)/3)
12. R = RealField(100, rnd='RNDU')
13. print(R(1)/3)
14. R = RealField(100, rnd='RNDZ')
15. print(R(1)/3)
```

Now, let us try to understand the working of the program. Line 1 creates a real number field R with 100 bits of precision using MPFR through SageMath. It tells SageMath to use high-precision floating-point numbers instead of the usual double precision. Line 2 computes and prints the value of 1/3 in that 100-bit precision field. Line 3 redefines R with higher precision (200 bits), and line 4 prints 1/3 again, now with greater accuracy, illustrating how precision can be increased without changing the computation itself. Line 5 adds 0.1 and 0.2 after converting them from machine floating-point numbers into the high-precision field, which may still carry tiny binary rounding errors from their original representation. Line 6 adds the numbers using string input "0.1" and "0.2", forcing SageMath to interpret them as exact decimals before conversion, thereby avoiding prior floating-point error. Line 7 prints the constant π computed to 200-bit precision using MPFR. Line 8 creates a 100-bit field with rounding mode 'round to nearest' (RNDN), and Line 9 prints 1/3 under this rounding rule. Lines 10 to 15 repeat the computation of 1/3 using different rounding modes: RNDD rounds downward, RNDU rounds upward, and RNDZ rounds towards zero. Together, these lines demonstrate how SageMath utilises MPFR's ability to control both precision and rounding behaviour, which is essential for reliable high-precision numerical work. Upon executing the program 'mpfr45.sage', the output shown in Figure 2 is obtained.

The last four lines in Figure 2 show the output produced using the four MPFR rounding modes. From this output alone, the distinction between RNDD and RNDZ may not be immediately apparent. The code snippet below clarifies the difference, demonstrating that these two modes behave differently only for negative numbers.

```
0.33333333333333333333333333333
0.33333333333333333333333333333333333333333333333333333333333333
0.30000000000000000000000000000000000000000000000000000000000000
0.30000000000000000000000000000000000000000000000000000000000000
3.14159265358979323846264338327950288419716939937510582097494
0.33333333333333333333333333333
0.33333333333333333333333333333
0.33333333333333333333333333334
0.33333333333333333333333333333
```

Figure 2: MPFR examples for floating-point arithmetic

```
R = RealField(50, rnd='RNDD')
print(R(0.9))
R = RealField(50, rnd='RNDZ')
print(R(0.9))
R = RealField(50, rnd='RNDD')
print(R(-0.9))
R = RealField(50, rnd='RNDZ')
print(R(-0.9))
```

Upon execution, the output shown below is produced. From this output, it is evident that rounding down and rounding towards zero differ when negative numbers are processed.

```
0.89999999999999
0.89999999999999
-0.90000000000001
-0.89999999999999
```

In addition to providing precise representations of integers and floating-point numbers, MPFR implements a wide range of advanced numerical functions that are particularly valuable in number-theoretic computations. The SageMath program 'functions46.sage' (line numbers included for clarity), shown below, demonstrates the use of several of these MPFR functions.

```
1. R = RealField(100)
2. x = -R(2).sqrt()
3. print(x)
4. print(abs(x))
5. print(x.sign())
6. print(R(100).exp())
7. print(x.prec())
8. x = R(3).sqrt()
9. print(x)
10. print(x.nextabove())
11. print(x.nextabove() - x)
```

Now, let us try to understand the working of the program. Line 1 creates a real-number arithmetic system with 100 bits of precision. Line 2 converts the integer 2 into a 100-bit real number and computes its square root inside that high-precision system. Then the result is negated. Line 3 prints the value of x, which is −√2, rounded to the number of decimal digits SageMath chooses to display. Line 4 prints the absolute value of x. Since x is negative, abs(x) removes the sign and prints a positive approximation of √2. Line 5 prints the sign of the number (-1 if negative, 0 if zero, and +1 if positive). Since x is −√2, the output is